Happy LDOC!

Function-Writing Practice & Algorithmic Complexity

Reminders

Re: Assignments:

- **EX09: Linked List Utility Functions** due tonight at 11:59pm
  - Late submission deadline: 4pm on April 30th (start of the final exam)
  - You can work with a partner (or on your own)!!

Re: Final Exam:

- **Hybrid Review Session** on Wednesday, April 29 @ 3:30-5:30pm in SN014 (Sitterson Hall, room 014)
- **4–7pm on Thursday, April 30**
  - Makeup final exam on Friday, May 1 in SN014
    - If you qualify for the makeup, please complete our internal form ASAP. You will get a confirmation email next week!

Apply to be a TA for COMP110 in Fall 2026! Deadline is May 3!

Function-writing practice

Write a function called **shape** that:

- Has one parameter, **input_data_table**, of type `dict[str, list[str]]`
- It should return a `dict[str, int]` where each key is a column name from **input_data_table** and its corresponding value is the number of elements in that column.

Example behavior:

```
>>> data = { "name": ["Alice", "Bob", "Carol"], "major": ["CS", "Math"]}

>>> shape(data)

{"name": 3, "major": 2}
```

Function-writing practice

```
def shape(input_data_table: dict[str, list[str]]) -> dict[str, int]:
    result: dict[str, int] = {}


    for column in input_data_table:
        result[column] = len(input_data_table[column])


    return result
```

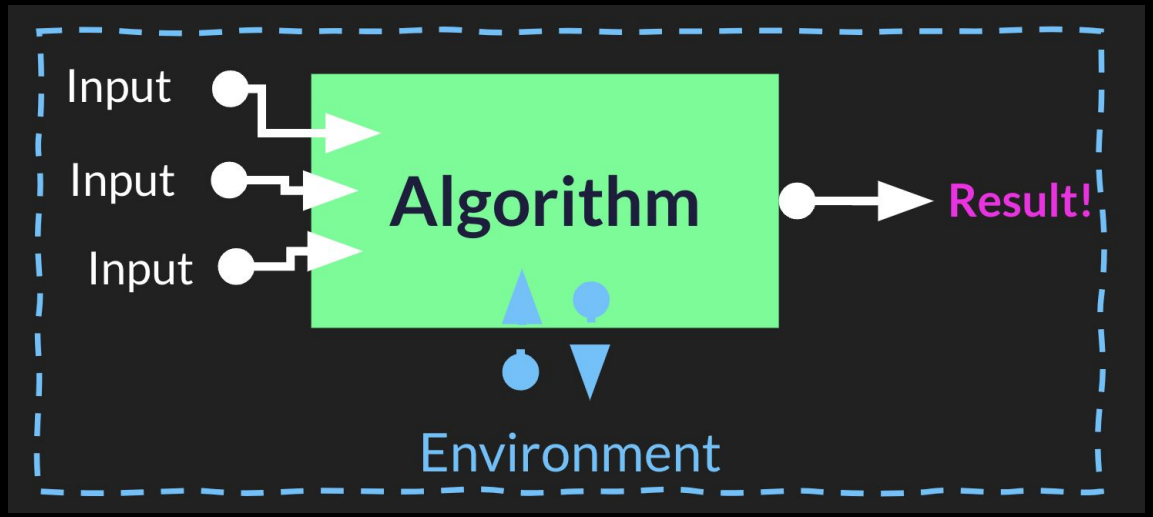
Review: Algorithms

Input is data given to an algorithm

An **algorithm** is a series of steps

An algorithm **returns** some **result**

An algorithm *may* be influenced by its **environment** and it *may* produce side-effects which influence its environment.



Running time: how long does an algorithm take to run?

- Empirical analysis: write the code and test how long it takes to run!
  - Weaknesses:
    - You have to write the code for the whole algorithm and run it to see how long it will take
    - Different computers with different specs will have different runtimes
- Rather than using empirical analysis, computer scientists commonly consider the **number of operations (steps)** an algorithm requires
  - 1 operation == 1 step

Runtime analysis: Best, average, and worst case

Best case (lower bound):

- Minimum number of operations (running time) required for the algorithm to execute

Average case:

- Average running time among several different inputs

Worst case (upper bound) ✨:

- The maximum running time given an input
  - How does the number of operations grow as an input grows?
- Important to understand how our algorithm will perform in the *worst* case
  - Prepare for the worst case. If an input ends up requiring fewer operations, great!!

Runtime analysis: order of growth →

The Complexity Hierarchy

From fastest to slowest — at large n

$O(1)$

Constant

$O(\log n)$

Logarithmic

$O(n)$

Linear

$O(n \log n)$

Linearithmic

$O(n^2)$

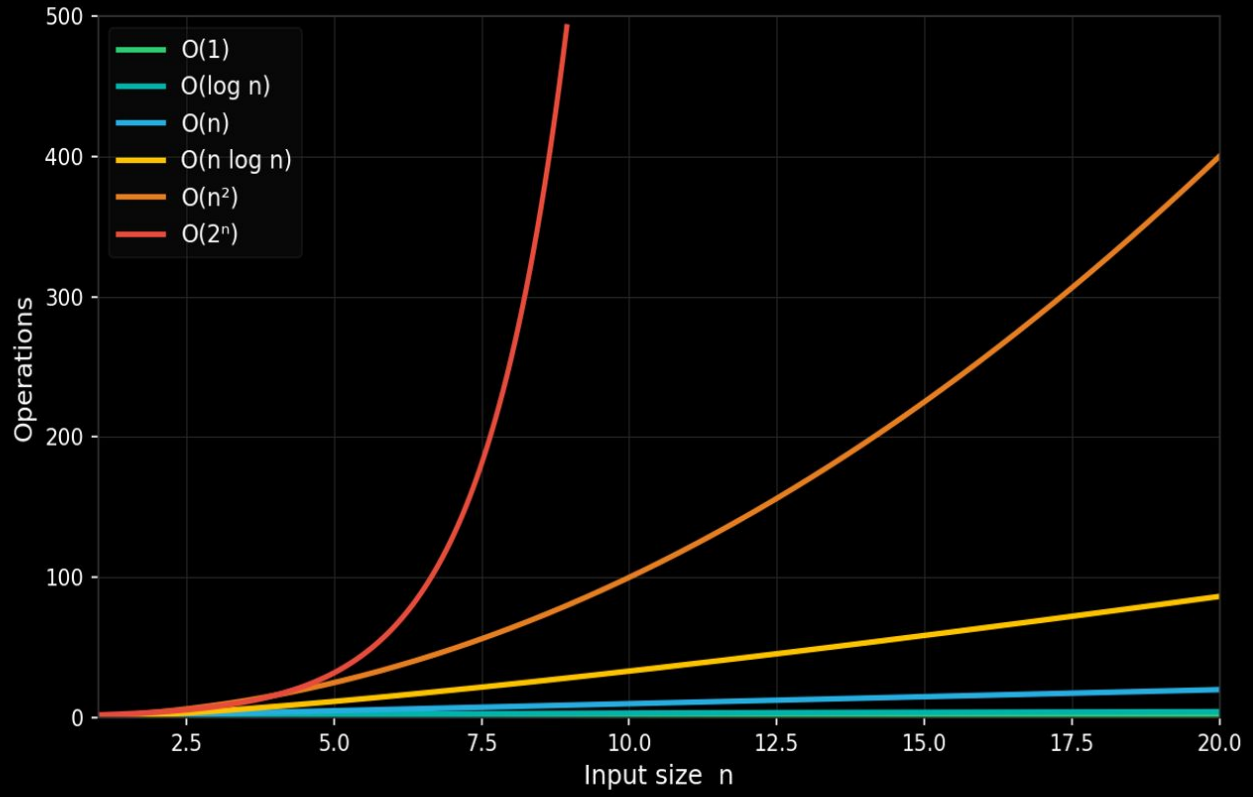
Quadratic

$O(2^n)$

Exponential

Growth Rates Visualized

How runtime scales with input size n



Key Takeaways

O(1) — Flat line — doesn't grow at all.

O(log n) — Very slow growth. Doubling n adds just 1 step.

O(n) — Linear — double n, double time.

O(n log n) — Slightly worse than linear. Most sorting.

O(n²) — Quadratic — dangerous for large n.

O(2ⁿ) — Explosive — unusable beyond tiny n.

Thank you for a great semester!

❤️, your COMP110 Team

P.S. Please complete your Course Evaluation;

it helps us improve the course!